

Problem Set 1

Harvard SEAS - Fall 2026

Due: Thurs 2026-09-17 (11:59pm)

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, earlier problem sets and earlier solutions sets in this semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments.

If you did not have any collaborators or external resources, please write 'none.' Please remember to select pages when you submit on Gradescope. A problem set on the border between two letter grades cannot be rounded up if pages are not selected OR collaborators not noted.

Your name:

Collaborators and External Resources:

No. of late days used on previous psets:

No. of late days used after including this pset:

1. (Asymptotic Notation)

- (a) (practice using asymptotic notation) Fill in the table below with “T” (for True) or “F” (for False) to indicate the relationship between f and g . For example, if f is $\Omega(g)$, the first cell of the row should be “T.” No justification necessary. Notice that some of the functions are the same as in Problem Set 0. Recall that, throughout CS1200, all logarithms are base 2 unless otherwise specified.

f	g	Ω	ω	Θ
$3(\log_2 n)^2$	$2(\ln n) \cdot (\ln n + 3)$			
$3n^3$	$ \{S \subseteq [n] : S \leq 5\} $			
$(n+1)^{n+1}$	$(n+1) \times n!$			
4^n	$(3 + (-1)^n)^n$			

- (b) (runtimes: $T^\equiv$ vs. T) Let $g : \mathbb{R}^{\geq 0} \rightarrow \mathbb{R}^{\geq 0}$ be a nondecreasing function, i.e. if $x_0 \geq x_1$, then $g(x_0) \geq g(x_1)$. (For example $g(n) = n^2$, $g(n) = n \log n$, or $g(n) = 2^n$.) Let $T^\equiv : \mathbb{N} \rightarrow \mathbb{N}$ and $T : \mathbb{R}^{\geq 0} \rightarrow \mathbb{N}$ be the runtimes of an algorithm A , as defined in Lecture 2.

- i. Consider the two statements:

$$“T^\equiv = \Omega(g) \implies T = \Omega(g)”$$

and

$$“T = \Omega(g) \implies T^\equiv = \Omega(g)”.$$

One of these statements is true and one is false. Prove the correct statement. For the false statement, give an example of a potential runtime $T^\equiv$ and a function g to demonstrate. (Hint: one of the pairs of functions in the table above may be helpful.)

- ii. (Optional question¹) Prove that $T^\equiv = O(g)$ iff $T = O(g)$. Thus the distinction between T and $T^\equiv$ does not matter when we are interested in “big-oh” runtime bounds, which are nondecreasing in most “natural” cases.

(Hint: Recall the formal definition of big-O notation: “there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that for all $n \geq n_0$, ...”. In the “only if” direction, you’ll need to find a constant c such that the big-O inequality holds for T . Think about how to construct this based on the formal relationship between $T^\equiv$ and T .)

¹This question will be graded only if you have seriously attempted all the other questions. It can only be used to increase your grade from an R to an R^+ .

2. (Understanding computational problems and mathematical notation)

Recall the definition of a *computational problem* from Lecture Notes 2.

Consider the following computational problem $\Pi = (\mathcal{I}, \mathcal{O}, f)$:

- $\mathcal{I} = \mathbb{N} \times \mathbb{N}^{\geq 2} \times \mathbb{N}$, where $\mathbb{N}^{\geq 2} = \{2, 3, 4, \dots\}$.
- $\mathcal{O} = \{(c_0, c_1, \dots, c_{k-1}) : k, c_0, \dots, c_{k-1} \in \mathbb{N}\}$
- $f(n, b, k) = \{(c_0, c_1, \dots, c_{k-1}) : n = c_0 + c_1b + c_2b^2 + \dots + c_{k-1}b^{k-1}, \forall i \ 0 \leq c_i < b\}$.

Here is an algorithm BC to solve Π :

```
1 BC( $n, b, k$ )
2 foreach  $i = 0, \dots, k - 1$  do
3    $c_i = n \bmod b$ ;
4    $n = (n - c_i)/b$ ;
5 if  $n == 0$  then return  $(c_0, c_1, \dots, c_{k-1})$ ;
6 else return  $\perp$ ;
```

- If the input is $(n, b, k) = (57, 12, 3)$, what does the algorithm BC return? Is BC's output a valid answer for Π with input $(57, 12, 3)$?
- Describe the computational problem Π in words. (You may find it useful to try some more examples with $b = 10$.)
- Is there any $x \in \mathcal{I}$ for which $f(x) = \emptyset$? If so, give an example; if not, explain why.
- For each possible input $x \in \mathcal{I}$, what is $|f(x)|$? ($|A|$ is the size of a set A .) Justify your answer(s) in one or two sentences.
- Let $\Pi' = (\mathcal{I}, \mathcal{O}, f')$ be the problem with the same $\mathcal{I}$ and $\mathcal{O}$ as Π , but $f'(n, b, k) = f(n, b, k) \cup \{(0, 1, \dots, k - 1)\}$. Does every algorithm A that solves Π also solve Π' ? (Hint: any differences between inputs that were relevant in the previous subproblem are worth considering here.) Justify your answer with a proof or a counterexample.

3. (Radix Sort) In Lecture, you saw the **SingletonBucketSort**, generalized to arrays of key-value pairs, and saw that it has running time $O(n + U)$ when the keys are drawn from a universe of size U . In this problem you'll study **RadixSort**, which improves the dependence on the universe size U from linear to logarithmic. Specifically, **RadixSort** can achieve runtime $O(n + n \cdot (\log U)/(\log n))$, so it achieves runtime $O(n)$ whenever $U = n^{O(1)}$.

RadixSort is constructed by using **SingletonBucketSort** as a subroutine several times, but on a smaller universe size b . Specifically, it turns each key from $[U]$ into an array of k subkeys from $[b]$ using the algorithm BC from Problem 2 above as a subroutine, and then iteratively sorts on each of the k subkeys. Crucially, **RadixSort** uses the fact that **SingletonBucketSort** can be implemented in a way that is *stable* in the sense that it preserves the order in the input array when the same key appears multiple times. Here is pseudocode for **RadixSort**:

```

1 RadixSort( $U, b, A$ )
   Input           : A universe size  $U \in \mathbb{N}$ , a base  $b \in \mathbb{N}$  with  $b \geq 2$ , and an array
                      $A = ((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$ , where each  $K_i \in [U]$ 
   Output          : A valid sorting of  $A$ 
2  $k = \lceil \log_b U \rceil$ ;
3 foreach  $i = 0, \dots, n-1$  do
4   |  $V'_i = \text{BC}(K_i, b, k)$  ;                               /*  $V'_i$  is an array of length  $k$  */
5 foreach  $j = 0, \dots, k-1$  do
6   | foreach  $i = 0, \dots, n-1$  do
7   |   |  $K'_i = V'_i[j]$ 
8   |   |  $((K'_0, (V_0, V'_0)), \dots, (K'_{n-1}, (V_{n-1}, V'_{n-1}))) =$ 
   |   |   SingletonBucketSort( $b, ((K'_0, (V_0, V'_0)), \dots, (K'_{n-1}, (V_{n-1}, V'_{n-1})))$ );
9 foreach  $i = 0, \dots, n-1$  do
10  |  $K_i = V'_i[0] + V'_i[1] \cdot b + V'_i[2] \cdot b^2 + \dots + V'_i[k-1] \cdot b^{k-1}$ 
11 return  $((K_0, V_0), \dots, (K_{n-1}, V_{n-1}))$ 

```

Algorithm 0.1: Radix Sort

(You can also read a description of Radix Sort in CLRS Section 8.3 for the case of sorting arrays of keys (without attached items) when U and b are powers of 2, albeit using different notation than us.)

- (a) (proving correctness of algorithms) Prove the correctness of **RadixSort** (i.e. that it correctly solves the **SortingOnFiniteUniverse** problem defined in Lecture).

Hint: You will need to use the stability of **SingletonBucketSort** in your argument. If it were replaced with an unstable implementation (or any other unstable sorting algorithm, such as **ExhaustiveSearchSort** with an unfortunate ordering on permutations), then the resulting algorithm would not be a correct sorting algorithm. For intuition, you may want to think about what happens when you sort a spreadsheet by one column at a time.

- (b) (analyzing runtime) Show that **RadixSort** has runtime $O((n + b) \cdot \lceil \log_b U \rceil)$. Set $b = \min\{n, U\}$ to obtain our desired runtime of $O(n + n \cdot (\log U)/(\log n))$. (This runtime analysis is outlined in CLRS, but you'd need to adapt it to our notation and slightly more general setting.)

4. (Reflection Question) There are a number of resources to support your learning in CS1200, such as Lecture, Ed, Office Hours, Sections, Hesterberg-Vadhan book, Recommended Readings, Collaboration with Classmates, Sender-Receiver Exercises. Which of these (or any others that come to mind) have you found most helpful so far and why? Are there ones that you should take more advantage of going forward? Do you have suggestions for how the course can make these more helpful to you?

Quick note on grading: Good responses are usually about a paragraph, with something like 7 or 8 sentences. Most importantly, please make sure your answer is specific to this class and your experiences in it. If your answer could have been edited lightly to apply to another class at Harvard, points will be taken off.

Note: As with the previous psets, you may include your answer in your PDF submission, but the answer should ultimately go into a separate Gradescope submission form.

5. Once you're done with this problem set, please fill out [this survey](#) so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses!